Multi-Robot Disaster Relief with Resource Allocation Optimization

Anshu Roja Selvamani and Darragh Buckley

I. Introduction

Disasters such as collapsed buildings, earthquakes, and chemical accidents among many others create environments that are too dangerous for human responders to navigate efficiently under time pressure. Autonomous multi-robot systems offer a scalable alternative. For instance, fleets of robots can be deployed simultaneously across a hazardous area, performing different tasks in parallel while minimising human exposure to danger.

This project develops a fully simulated five robot disaster relief system in the Webots robotics simulator. The core research question is whether a sophisticated auction-based allocator such as Consensus Bundle Based Algorithm produces measurably better mission outcomes than a simple greedy nearest robot baseline, across metrics including task completion rate, average completion time, battery consumption, and robustness to dynamic events.

II. Project Scope and Progress

The original project scope proposed three interconnected components: (1) multi-robot task allocation comparing Greedy and CBBA, (2) energy-aware A* path planning on a discretized grid, and (3) dynamic reallocation triggered by battery, stuck-detection, and priority events. All three components have been implemented and are running in simulation. The scope is unchanged. The remaining work focuses on extended simulation runs, stress-testing with more dynamic tasks, and final write-up. There are still some minor errors in path planning that we will have to work on. The prioritisation is a little error prone currently. We will continue to develop and test this. Concretely, the following milestones are complete as of this report: the Webots environment is built with thirteen obstacle objects; all five robots move, drain battery, and charge autonomously. The A* planner routes safely around most obstacles. Both Greedy and CBBA allocators assign tasks correctly with no duplicate assignments.Dynamic task cylinders spawn and are removed on completion. In addition, preliminary results have been exported and visualised.

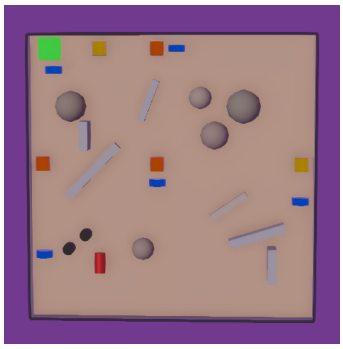

Fig. 1. Created world with obstacles, tasks and robots in action

III. System Architecture

The system follows a centralised supervisor-worker architecture implemented in Python within the Webots robotics simulator. The codebase is organised into two controller directories containing six source files, with a clear separation between coordination, robot behaviour, and algorithm logic.

A. Overall Structure

The Supervisor Controller (supervisor_controller.py) acts as the central coordinator. It maintains global state for all five robots and the task pool, and orchestrates the simulation loop on a fixed timestep. Each iteration follows a fixed pipeline: move active robots, check task deadlines, evaluate reallocation triggers, spawn dynamic tasks if due, run the selected allocation algorithm, apply new assignments via A* path planning, and log metrics. The Robot Controller (robot_controller.py) is a lightweight per-robot process that tracks local battery state and reports status back to the supervisor via Webots customData JSON messages. Movement is handled entirely by the supervisor through direct translation updates, so the robot controller acts as a state reporter rather than an autonomous navigator.

B. Modularity and Algorithm Switching

Modularity and Algorithm Switching: The three core algorithms (A* path planning, Greedy allocation, and CBBA allocation) are implemented as separate, self-contained Python modules (astar.py, greedy.py, cbba.py) imported by the supervisor. The active allocator is selected by a single boolean flag at the top of the supervisor file, allowing both algorithms to be run in identical environments for fair comparison without any other code changes. A sixth file, results.py, handles post-simulation visualisation by loading the JSON output from both runs and generating comparison plots using matplotlib.

C. Simulation Environment

The Webots world defines a 10 x 10 metre arena enclosed by boundary walls, containing thirteen obstacle objects including standing walls, collapsed slabs, rocks, barrels, and debris. Five robots are placed at fixed starting positions. A charging station is located in the upper-right corner of the arena. Tasks are represented visually as colored cylinders, color-coded by priority level.

D. Communication and Data Flow

The supervisor communicates with robots via Webots customData fields, reading battery and state reports each timestep. Robot positions are read directly from Webots node translations. Task and robot state are maintained in in-memory Python dictionaries within the supervisor process. At the end of the simulation (timestep 10,000), all recorded metrics are exported to a JSON file for analysis.

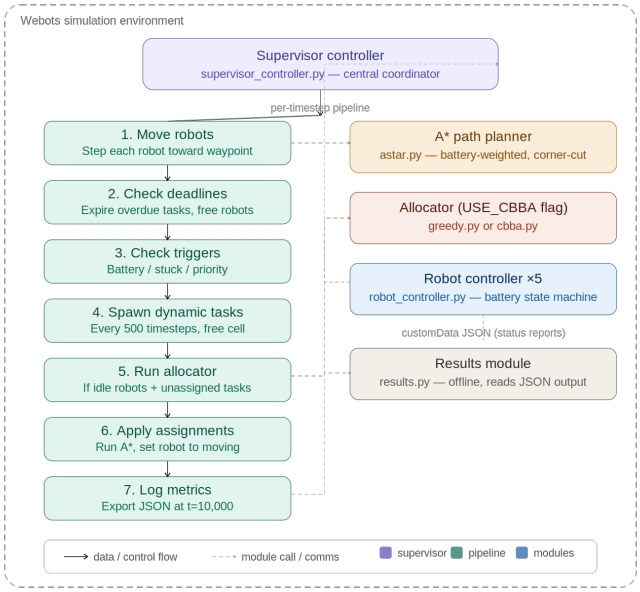

Fig. 2. System Overview Diagram

IV. Implemented Approach

The implemented approach combines task allocation, energy-aware path planning, and dynamic reallocation to address the challenges of coordinating multiple robots in a constrained and time-sensitive disaster environment. The problem is formulated as a multi-agent task allocation problem in which a set of robots

$$R = \{r_1, r_2, \ldots, r_n\}$$

must be assigned to a set of tasks

$$T = \{t_1, t_2, \ldots, t_m\},$$

where each task is characterized by its spatial location, priority level, and deadline, and each robot is constrained by its battery capacity and current state. The objective is to maximize task completion while minimizing completion time and energy consumption.

We compared the Greedy approach and the Consensus-Based Bundle Algorithm (CBBA) approach. The three major algorithms we implemented were are A* path planning, Greedy task allocation, and CBBA.

The Greedy allocator serves as the baseline method. Tasks are first sorted in descending order of priority. For each task in this ordering, the nearest idle robot with battery above 30% is assigned to the task. This is a nearest-neighbor heuristic with $O(|T| \cdot |R|)$ complexity per round.

A* Path Planning: The Webots world is discretized into a 10×10 grid of $1\,\mathrm{m}^2$ cells, covering the $-5\,\mathrm{m}$ to $+5\,\mathrm{m}$ world bounds. Obstacle positions are derived directly from the `.wbt` world file translations and mapped to grid cells. The path cost function penalises longer routes more heavily as battery drains, encouraging energy-conservative routing. The path cost is defined as

$$g(n) = g(\text{parent}) + \text{move_cost}\left(1 + \beta\left(1 - \frac{\text{battery}}{100}\right)\right),$$

where $\text{move_cost} = 1.0$ for cardinal moves and $\text{move_cost} = 1.414$ for diagonal moves, and where $\beta = 1.2$ controls the battery penalty. Diagonal moves are additionally blocked when either orthogonal neighbour is an obstacle cell which we researched and found is a corner-cutting prevention rule that stops robots from slipping through diagonal gaps between obstacles. As for CBBA, it operates in two phases. In Phase 1 which is bundle building, each robot independently builds a bundle of up to three tasks by greedily selecting the task with the highest bid score at each step. The bid score for robot r and task t is defined as

$$\begin{aligned}\text{score}(r,t) = 5.0 \cdot \text{priority}(t) \\ + 0.3 \cdot \frac{\text{battery}(r)}{100} \\ - 0.1 \cdot \text{dist}(r,t) \\ - 0.5 \cdot |\text{bundle}(r)|\end{aligned} \tag{1}$$

In this case, task priority dominates the score through the weight 5.0, ensuring that high-priority tasks are strongly preferred regardless of distance. The bundle penalty of 0.5 per task already assigned discourages a single robot from hoarding all available tasks and promotes a more balanced allocation across the team.

In the second phase, referred to as consensus, robots iteratively compare bids for contested tasks. For each task claimed by multiple robots, the robot with the highest bid retains ownership, while all losing robots remove that task from their bundles. This process repeats for up to 100 iterations or until no bundle changes occur, indicating convergence. After convergence, a greedy fallback assigns any remaining unclaimed tasks to idle robots, ordered by descending battery level.

The final component of the system is dynamic reallocation. The supervisor monitors three trigger conditions at every simulation timestep. First, a battery trigger activates when a

robot's charge falls below 30%, causing its current task to be released back into the task pool. Second, a stuck trigger activates when a robot has not moved more than 0.005 m over 200 consecutive timesteps, at which point the task is released and the robot is reset to the idle state. Third, a priority trigger activates whenever a newly spawned dynamic task has priority level 3, causing immediate reallocation. In all cases, the supervisor releases the task before invoking the allocator, ensuring that no double-release occurs.

Dynamic tasks are spawned every 500 timesteps at random free grid cell locations. These locations are sampled from a precomputed list of valid cell centers, guaranteeing that newly created tasks never appear inside obstacle regions.

V. Preliminary Results

Both algorithms were run for over 10,000 simulation timesteps in identical Webots environments with five robots. The results are summarized in Table I and visualized in Fig 3.

TABLE I
SIMULATION RESULTS OVER 10,000 TIMESTEPS

Metric	Greedy	CBBA
Task Completion Rate	96.7%	96.7%
Avg Completion Time (timesteps)	63.17	52.45
Avg Battery per Task (%)	0.51%	0.42%
Reallocation Events	0	0
Time Lost to Reallocation	0	0
Robot Battery Failures	0	0
Tasks Expired	0	0

CBBA outperforms the Greedy baseline on both key efficiency metrics. The average task completion time is reduced by approximately 17.0%, decreasing from 63.17 to 52.45 timesteps. Similarly, the average battery consumption per task is reduced by 17.6%, decreasing from 0.51% to 0.42%. Both algorithms achieve a task completion rate of 96.7%, with a single task remaining incomplete in each run due to end-of-simulation timing. No reallocation events were triggered in either run, indicating that the current task density and battery drain parameters maintain system stability without requiring emergency intervention.

Fig. 3. Results generated comparing both algorithm runs Greedy vs CBBA

The CBBA advantage in completion time stems from its bundle-building phase: by reasoning about multiple tasks simultaneously and scoring them against competing robots, CBBA produces globally more coherent assignments where robots are sent to clusters of nearby tasks rather than the globally nearest single task. The Greedy algorithm's completion time distribution shows a longer tail that is up to 258 timesteps vs. 200 for CBBA, confirming that Greedy makes locally optimal but globally suboptimal decisions for distant tasks.

The absence of reallocation events is consistent with the simulation design: battery drain is gradual ((0.008% per timestep) and tasks are spread across a 10×10 m area, so robots complete most tasks well before reaching the 30% battery threshold. Future runs with higher task density, faster drain, or more dynamic obstacles are expected to exercise the reallocation system. Also if we were to use larger number of robots in the testing runs, CBBA would prove to show a bigger difference in efficiency.

VI. Future Plan

While the current implementation demonstrates a functional and validated multi-robot disaster relief system, several directions remain for the final submission.

The most immediate priority is stress-testing the reallocation system. In the current preliminary runs, no reallocation events were triggered because battery drain is gradual and task density is moderate. To properly evaluate the three-trigger reallocation mechanism, future runs will use a higher battery drain rate, a faster dynamic task spawn interval, and a deliberately induced stuck scenario where one robot is blocked at timestep 500. These conditions are expected to force battery, stuck, and priority triggers to fire, allowing a meaningful comparison of how quickly Greedy and CBBA recover task coverage after an emergency reallocation event.

A second direction is scalability testing. The current implementation runs with five robots. The CBBA consensus phase has O(R × T × I) complexity where I is the number of iterations, meaning convergence time grows with fleet size. We plan to run trials with eight robots to determine whether CBBA's performance advantage is maintained or whether the additional consensus overhead starts to erode gains relative to Greedy. This will directly address the scalability limitation acknowledged in the project proposal.

A third direction is refining the path planning edge cases. As noted in Section II, minor issues remain in edge-case navigation scenarios. Specifically, robots whose starting position is very close to an obstacle boundary sometimes receive A* paths that require backing out of a partially obstructed cell before proceeding. While the BFS fallback handles these cases without crashing, the resulting paths are occasionally longer than necessary. A planned fix is to widen the obstacle buffer by snapping all robot start positions to the nearest cell center at assignment time rather than using raw translation coordinates.

The prioritization logic also requires refinement. Currently, CBBA's bid score formula weights priority at 5.0 but this was

tuned manually. In scenarios with many high-priority tasks arriving simultaneously, multiple robots may all bid heavily on the same small set of urgent tasks, leaving lower-priority tasks unattended for long periods. An adaptive priority weight that scales with the number of unassigned high-priority tasks relative to available robots would address this.

Finally, three independent trials per algorithm with fixed random seeds will be run to produce averaged metrics with standard deviations, enabling statistically grounded conclusions rather than single-run comparisons. Results will be reported across all three evaluation scenarios.